SQL Alchemy

SQLAlchemy ORM pour les intégrateurs d'IA 1
Introduction 1
Pourquoi utiliser SQLAlchemy dans des projets IA ? 1
Vocabulaire Merise vs SQLAlchemy 1
Définir une table avec SQLAlchemy 2
Initialiser la base de données 3
Manipuler les données avec l'ORM 3
Ajouter des données 3
Lire des données 3
Modifier / Supprimer 4
Contraintes relationnelles et intégrité 4
Clé primaire 4
Clé étrangère 4
Relation 1-N (Merise) 4
Intégration avec FastAPI (Depends + session par requête) 5
Migrations avec Alembic 6
Initialiser Alembic dans un projet 6
Configurer la base de données 6
Créer une migration 7
Appliquer une migration 7
Exemple concret : ajout d'un champ date dans Commande 8

SQLAlchemy ORM pour les intégrateurs d'IA

Introduction

SQLAlchemy est une bibliothèque Python qui permet de communiquer avec une base de données relationnelle (PostgreSQL, SQLite, MySQL, etc.) en utilisant du code Python orienté objet. Son ORM (Object-Relational Mapper) permet de représenter les tables SQL sous forme de classes Python.

Pourquoi utiliser SQLAlchemy dans des projets IA ?

- Suivre les jeux de données, les expériences, les utilisateurs, les journaux...
- Structurer des bases de données relationnelles facilement
- Automatiser le stockage des résultats intermédiaires d'entraînement ou d'évaluation
- Manipuler les données comme des objets Python tout en garantissant l'intégrité relationnelle

Vocabulaire Merise vs SQLAlchemy

Terme Merise	SQLAlchemy	Description
Entité	Classe Python	Représente une table
Attribut	`Column`	Représente une colonne dans la table
Identifiant	`primary_key=True`	Clé primaire
Association	`ForeignKey` + `relationship`	Lien entre deux entités

Définir une table avec SQLAlchemy

On commence par définir une classe Python qui hérite de Base. Cette classe représente une table SQL.

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import declarative_base, relationship

Base = declarative_base()

class Client(Base):
    __tablename__ = 'clients'
    # clé primaire
    id = Column(Integer, primary_key=True)
    nom = Column(String)
    # relation 1-N
    commandes = relationship("Commande",
back_populates="client")

class Commande(Base):
    __tablename__ = 'commandes'
    id = Column(Integer, primary_key=True)
    montant = Column(Integer)
    # clé étrangère
    client_id = Column(Integer, ForeignKey('clients.id'))
    client = relationship("Client",
back_populates="commandes")
```

Les relations bidirectionnelles (avec `back_populates`) permettent d'accéder facilement aux données dans les deux sens (Client → Commandes, Commande → Client).

Initialiser la base de données

Une fois les classes définies, il faut créer une connexion à la base et générer les tables.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

# Connexion à une base PostgreSQL (exemple)
engine =
create_engine("postgresql://user:password@localhost/ma_base")
Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)
session = Session()
```

Manipuler les données avec l'ORM

Ajouter des données

```
nouveau_client = Client(nom="Alice")
session.add(nouveau_client)
session.commit()

commande = Commande(montant=200, client=nouveau_client)
session.add(commande)
session.commit()
```

Lire des données

```
# Lire toutes les commandes d'un client
client = session.query(Client).filter_by(nom="Alice").first()
for cmd in client.commandes:
    print(cmd.montant)
```

Modifier / Supprimer

```
# Modifier un montant
commande.montant = 300
session.commit()

# Supprimer une commande
session.delete(commande)
session.commit()
```

Contraintes relationnelles et intégrité

Clé primaire

Chaque table doit avoir une clé primaire unique :

```
id = Column(Integer, primary_key=True)
```

Clé étrangère

Permet de lier une table à une autre :

```
client_id = Column(Integer, ForeignKey('clients.id'))
```

Relation 1-N (Merise)

Un client peut avoir plusieurs commandes, mais chaque commande appartient à un seul client.
Utilisation de `relationship()` pour faciliter l'accès :

```
commandes = relationship("Commande", back_populates="client")
```

Intégration avec FastAPI (Depends + session par requête)

Dans un projet IA, on veut souvent exposer une API REST qui accède à une base de données via SQLAlchemy. FastAPI permet de gérer proprement les connexions à la base en ouvrant une session pour chaque requête.

```
# db.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base

SQLALCHEMY_DATABASE_URL = "sqlite:///./test.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL,
connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Ensuite, dans un routeur FastAPI, on peut injecter la session avec `Depends(get_db)` :

```python
# main.py
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from db import get_db, Base, engine
from models import Client

app = FastAPI()
Base.metadata.create_all(bind=engine)

@app.get("/clients")
def lire_clients(db: Session = Depends(get_db)):
    return db.query(Client).all()
```

Ce modèle garantit qu'une session est ouverte et fermée proprement pour chaque appel HTTP, ce qui évite les fuites de connexion et les conflits.

Migrations avec Alembic

Lorsqu'une application évolue, il est nécessaire de modifier la structure de la base de données (ajout de colonnes, modification de types, etc.). Alembic est l'outil officiel pour gérer les **migrations de schéma** avec SQLAlchemy.

Initialiser Alembic dans un projet

```
alembic init alembic
```

Cela crée un dossier `alembic/` avec les fichiers de configuration.

Configurer la base de données

Dans `alembic.ini`, configure l'URL de la base de données :

```
sqlalchemy.url = sqlite:///./test.db
```

Et dans `alembic/env.py`, importe les modèles SQLAlchemy :

```
from myapp.models import Base

target_metadata = Base.metadata
```

Créer une migration

Une fois que tes modèles ont changé, crée un script de migration :

```
alembic revision --autogenerate -m "ajout champ date à
commande"
```

Cela crée un fichier dans `alembic/versions/` avec les instructions SQL générées automatiquement.

Appliquer une migration

```
alembic upgrade head
```

Cela applique les modifications à la base.

Alembic permet de gérer facilement les versions de ta base sans tout recréer à chaque changement.

Exemple concret : ajout d'un champ date dans Commande

1. On modifie le modèle :

```
from sqlalchemy import Column, Integer, String, ForeignKey,
Date

class Commande(Base):

    __tablename__ = 'commandes'

    id = Column(Integer, primary_key=True)

    montant = Column(Integer)

    date = Column(Date)  # Nouveau champ

    client_id = Column(Integer, ForeignKey('clients.id'))
```

2. On génère la migration automatiquement :

```
alembic revision --autogenerate -m "Ajout champ date dans
commande"
```

3. On vérifie le fichier généré dans `alembic/versions/` puis on applique :

```
alembic upgrade head
```

Cela ajoute une nouvelle colonne `date` dans la table `commandes` sans avoir à recréer toute la base.